Edxso AI Engineer Intern - Assignment 2

Build the Scholarship Intelligence Crawler

Real-World Problem

Refer to the existing **Atlas Funding** product:

Atlas Funding — Product Prototype

Atlas is designed to help students discover education funding opportunities and match them against their eligibility. The product currently envisions a repository where every funding opportunity is normalised into a common schema, with a continuous crawler maintaining that repository.

Your challenge

Build the intelligence engine that keeps this repository continuously updated with authentic scholarship opportunities for Indian students.

You are **not** being asked to design a crawler or explain how you would build one.

You have to build a working version.

1. What you need to build

Create an automated system that:

Discovers → Crawls → Extracts → Verifies → Scores → Stores → Updates

scholarship information from the internet.

The system should find scholarships available to **Indian students** and convert information from different websites into a consistent database.

Your system should work with sources such as:

- Government scholarship portals
- Central and State Government departments
- UGC / AICTE / ministries
- Universities and educational institutions

- Corporate CSR scholarship programmes
- Foundations and trusts
- Other legitimate scholarship providers

The Atlas prototype specifically envisages Government, Universities, Corporate CSR, International and NGO/Trust funding sources within a common repository.

2. The most important requirement: AUTHENTIC INFORMATION

We care more about **accuracy than quantity**.

A scholarship should not be considered verified merely because an AI model found it on the internet.

For every scholarship, your system must identify and retain the **official/primary source**.

Examples:

- Government website
- Official ministry/department website
- Official university website
- Official organisation/provider website
- Official scholarship portal

Aggregator websites, blogs and articles can be used for **discovery**, but they should not be treated as the authoritative source.

Every verified scholarship must have:

- Official source URL
- Application URL, where available
- Source type
- Date last verified
- Evidence used for verification

3. Extract the scholarship

Your crawler should convert unstructured information into a structured scholarship record.

At minimum capture:

- Scholarship name
- Provider
- Official source
- Application URL
- Scholarship amount / benefit
- Eligibility
- Academic requirements
- Course / education level
- Income criteria
- Age criteria
- Gender criteria, if applicable
- Category criteria, if applicable
- Domicile / state requirements
- Institution requirements
- Opening date
- Closing date
- Documents required
- Selection process
- Renewal requirements
- Current status

The Atlas concept uses a universal schema and machine-readable eligibility logic, including income, merit, category, domicile, disability, defence, sports and research conditions.

You do not have to reproduce Atlas's complete 38-field schema, but your schema should be sufficiently detailed to demonstrate that your crawler can support this type of system.

4. Build a verification engine

This is where the assignment becomes an **AI engineering problem**, rather than simply a web-scraping exercise.

For every scholarship, calculate a **confidence score**.

Example:

> **Scholarship:** ABC Scholarship
> **Confidence:** 98.4%
> **Status:** VERIFIED

The confidence score should be based on actual evidence.

For example:

- Is the source official?
- Is the scholarship currently present on the official source?
- Is there an official application URL?
- Can the eligibility criteria be directly supported?
- Can the deadline be directly supported?
- Is the information current?
- Are there conflicting sources?
- Was the information extracted consistently?
- Can the system trace important fields back to source evidence?

Critical rule

Do not ask an LLM to simply generate a confidence number.

You must design a methodology that produces the score.

A scholarship should only be labelled:

VERIFIED

when the system's confidence is **95% or above**.

Anything below that should be:

REVIEW REQUIRED

5. Prevent hallucinations

Your system must never invent scholarship information.

If the official source does not mention an income limit:

> Income limit: **Not specified**

not:

> Income limit: ₹5 lakh

Every important field should ideally be traceable to source evidence.

For example:

Deadline: 31 August 2026

should allow us to trace:

Database → Official Source → Evidence → Extracted Value

This is one of the most important evaluation criteria.

6. Make it an AUTO CRAWLER

This cannot be a one-time scraper.

The system should be capable of running repeatedly.

For example:

Run 1

> Scholarship A discovered
> Scholarship B discovered
> Scholarship C discovered

Run 2

> Scholarship A unchanged
> Scholarship B deadline changed
> Scholarship C no longer available
> Scholarship D newly discovered

The database should update accordingly.

7. Change detection

Demonstrate that your system can detect changes.

For example:

Previous

Deadline:

31 August 2026

New crawl

Deadline:

15 September 2026

Your system should identify:

    CHANGE DETECTED

and retain:

- Old value
- New value
- Date detected
- Source
- Evidence

Do not simply overwrite the old information.

8. Stale / expired scholarships

Your system should also identify scholarships that are:

- Expired
- No longer available
- Removed from the official source
- Unable to be re-verified
- Contradicted by a newer official notification

Use appropriate statuses such as:

`ACTIVE`

`EXPIRING_SOON`

`EXPIRED`

`REVIEW_REQUIRED`

`NO_LONGER_VERIFIABLE`

9. Build a working database

Use a free database technology.

For example:

- SQLite
- PostgreSQL
- Supabase free tier
- Firebase free tier

We should be able to inspect the database and see the actual records generated by your crawler.

A scholarship record should look approximately like:

Field	Example
Scholarship	ABC Scholarship
Provider	XYZ Foundation
Amount	₹50,000
Eligibility	Structured criteria
Deadline	30 Sept 2026

Official Source	URL
Application URL	URL
Status	ACTIVE
Confidence	98.2%
Last Verified	19 Aug 2026
Evidence	Source evidence

10. Build a simple working interface

Create a basic dashboard so we can see what your system has produced.

It should show things such as:

Scholarship Intelligence

Total discovered

Verified

Review required

Active

Expired

Recently updated

Average confidence

Then provide a searchable list of scholarships.

When we open a scholarship, we should be able to see:

Scholarship Details

Name

Provider

Amount

Eligibility

Deadline

Official Source

Application Link

Status

Confidence Score

Why this score?

Last verified

Change history

Source evidence

11. Minimum working output

Your crawler must produce **at least 20 real scholarship records**.

Minimum requirements:

- **20+ real scholarships**
- **15+ verified against primary/official sources**
- **10+ with confidence ≥95%**
- At least **3 different source types**
- At least **2 examples of change detection**
- At least **2 examples of expired/stale detection**

We will manually inspect a sample of your records.

Fake, fabricated or unsupported scholarship information will be treated as a serious failure.

12. Discovery should not be completely hard-coded

You may start with a small number of seed sources.

However, do not simply create:

URL 1 → scraper

URL 2 → scraper

URL 3 → scraper

...

and call that an intelligent crawler.

We want to see your approach to:

Discovery → Source classification → Crawling → Extraction → Verification

The system should demonstrate that it can discover scholarship opportunities beyond a fixed list of manually entered records.

13. Free tools only

You must use **free/open-source tools or genuinely free tiers**.

Possible technologies include:

- Python
- Scrapy
- BeautifulSoup
- Playwright
- Selenium
- Hugging Face
- Ollama
- Open-source LLMs
- Open-source embedding models
- FAISS
- SQLite
- Supabase free tier
- FastAPI
- Streamlit

You are free to choose your own stack.

Paid APIs and paid scraping services are not permitted.

14. What we expect you to submit

A. Working application

Preferably a live URL.

Otherwise provide clear local installation instructions.

B. GitHub repository

Include:

- Source code
- README
- Requirements
- Database/schema
- Configuration
- Sample data
- Setup instructions

C. Working demonstration

A short demonstration showing:

Crawler starts

↓

Discovers scholarship

↓

Extracts information

↓

Identifies official source

↓

Verifies information

↓

Generates confidence

↓

Stores record

↓

Displays scholarship

↓

Runs again

↓

Detects changes

D. Short technical note

Maximum **3 pages** covering:

- Architecture
- Technology choices
- Discovery methodology
- Extraction methodology
- Verification methodology
- Confidence-score methodology
- Anti-hallucination approach
- Change detection

Do not spend the submission explaining something that you haven't built.

15. Evaluation

Criteria	Weight
Working crawler & automation	20%

Authenticity & official-source verification	25%
Accuracy of extracted information	20%
Confidence / verification engine	15%
Change & stale-data detection	10%
Working UI / product output	5%
Code quality	5%
Total	**100%**

Automatic failure

The assignment fails if:

- It is only a design/architecture document
- It is only a UI mockup
- The crawler does not actually work
- Scholarship information is fabricated
- Official sources cannot be traced
- Confidence scores are arbitrary
- The system simply copies data from aggregators
- Paid services are used

The real test

We are **not** primarily testing whether you know Scrapy, BeautifulSoup, FastAPI, LangChain or a particular LLM.

We are testing whether you can take a real-world problem and build a system that produces **trustworthy AI-generated output**.

At the end, we should be able to ask you one question:

"Show us what your crawler found."